

Core Concepts — LangChain

Track 16.3 – [LangChain](#) · [16-ai-tech-stack/tracks/langchain/notes/01-core-concepts.md](#)

Written against **langchain 1.4.0 / langchain-core 1.6.3** (verified in this repo's venv, Python 3.14).
LangChain 1.0 shipped 2025-10-20 and moved a lot of furniture. If a tutorial you find imports `LLMChain`, `ConversationBufferMemory`, `initialize_agent` or `RetrievalQA`, it is describing v0. Those still exist — in a package called `langchain-classic`.

1. What LangChain actually is

Three things wearing one name:

1. **A provider-neutral interface layer.** One `ChatModel` API over Anthropic, OpenAI, Google, Bedrock, Ollama and ~100 others; one `Embeddings`, one `VectorStore`, one `Document`, one message format.
2. **An adapter catalogue.** Hundreds of loaders, stores, retrievers and tool wrappers implementing those interfaces so you don't write the glue.
3. **An agent harness.** `create_agent` — a model-calls-tools loop with middleware, built on LangGraph.

That's the whole product. Everything else is composition.

When to use it

- You will swap models or providers, and you don't want that to be a rewrite.
- You want retrieval, tool calling, streaming, retries and tracing without hand-rolling five protocols.
- You want LangSmith traces essentially for free (see `../langsmith/`).

When NOT to use it — read this part twice

"Just call the API." If your app is *one prompt, one model, one provider, parse the answer*, the Anthropic SDK does it in ~10 lines and those 10 lines have no release notes. A framework around a single API call buys you a `pip` tree, an upgrade obligation, and an extra stack frame between you and your bug.

The abstraction cost is real. A failing `prompt | model | parser` gives you a traceback through `RunnableSequence.invoke` → `RunnableBinding` → `BaseChatModel.generate` → the provider client, with your prompt buried in there as a serialized `ChatPromptValue`. "Generic pipeline framework" and "legible stack traces" are in tension and the framework picked the first.

Debugging opacity — with a cure you must actually apply: a tracer (`LangSmith`, `set_debug(True)`, or `.astream_events()`). Running a non-trivial LangChain app untraced is the #1 reason teams conclude "LangChain is a black box." It isn't. It is opaque *by default*, and turning the lights on is your job.

Churn. 2024-06 integration split, 2024-10 "LangGraph is the answer" deprecation wave, 2025-10 the 1.0 namespace amputation, plus `create_react_agent` → `create_agent`. That is good engineering — they kept moving toward the right design — but you paid for it in migration guides. Budget for it.

My take: use `langchain-core` almost always (the interfaces are the good part and they are stable), use `langchain` when you want the agent harness, use `langchain-community` with your eyes open, and reach for the raw provider SDK when the thing you are building is genuinely one API call. The worst outcome is a five-layer `RunnableSequence` that wraps a single `messages.create()`.

2. The package split, and why it happened

```
langchain-core      interfaces + Runnable + LCEL + messages + prompts + parsers
                    |
                    | ZERO heavy deps. This is the part you actually depend on.
                    |
+-- langchain       create_agent, middleware, init_chat_model, re-exports
                    |
                    | (built on langgraph)
                    | +-- langgraph  the runtime: graphs, state, checkpointers, durable execution
                    |
+-- langchain-anthropic / -openai / -google-genai / ...  one package per provider
+-- langchain-text-splitters                             chunking, no model deps
+-- langchain-community                                 the long tail, community-maintained
+-- langchain-classic                                   v0 chains, retrievers, indexing, hub
```

Why it happened. By mid-2024 the monolith carried 700+ integrations, so one integration's broken dependency bumped the whole ecosystem and one provider's breaking change forced a `langchain` release. The split gives each provider its own cadence and gives `langchain-core` the thing it needed most: a tiny dependency surface and a real stability promise.

The stability story, honestly stated:

Package	Stability	What that means for you
langchain-core	Strong. Breaking changes only on a major version.	Depend on it freely. $\geq 1.6, < 2$.
langchain	Strong since 1.0, but 1.0 itself was a namespace amputation.	Pin a major. Read the changelog.
provider packages	Follows the provider. Moves when the API moves.	Pin a minor if you care.
langchain-community	Weakest. Community-maintained, uneven test coverage.	Read the source before you ship it.
langchain-classic	Frozen-ish. It exists so you don't have to migrate today.	A migration you owe yourself, not a home.

The v1 namespace reduction. `langchain` now holds only `agents`, `messages`, `tools`, `chat_models`, `embeddings` (plus `mcp`, `rate_limiters`) — verify it yourself with `python -c "import pkgutil, langchain; print([m.name for m in pkgutil.iter_modules(langchain.__path__)])"`. Legacy chains, `langchain.retrievers`, the indexing API and `langchain.hub` moved to `langchain-classic`. In a 2023 blog

post, essentially every import needs rewriting. Source:
<https://docs.langchain.com/oss/python/migrate/langchain-v1>

3. Runnable — the one interface

Everything composable in LangChain implements `Runnable`. Prompts, chat models, output parsers, retrievers, tools, and the objects made by `|`. The contract:

Method	Returns	Notes
<code>invoke(input, config)</code>	one output	the only method you must implement
<code>batch(inputs, config)</code>	list of outputs	thread pool; <code>config={"max_concurrency": n}</code>
<code>stream(input, config)</code>	iterator of chunks	falls back to a single chunk if a step can't stream
<code>ainvoke</code> / <code>abatch</code> / <code>astream</code>	async twins	free, derived, or natively overridden
<code>astream_events(input)</code>	iterator of typed events	the debugging API

And the decorators that make it production-shaped:

```
chain.with_retry(retry_if_exception_type=(RateLimitError,), stop_after_attempt=3)
chain.with_fallbacks([cheaper_model])
chain.with_config(run_name="faq_answer", tags=["prod"]) # names your LangSmith spans
chain.configurable_fields(temperature=ConfigurableField(id="temperature"))
chain.bind(stop=["\n\n"]) # freeze kwargs onto every call
```

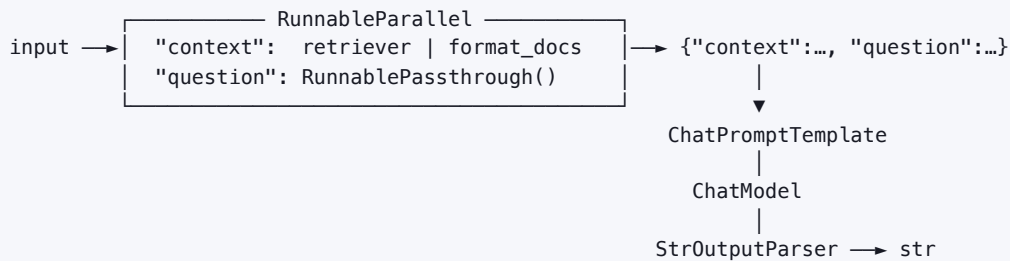
The insight worth internalising: implement `_generate`, `_stream` and `_llm_type` on a `BaseChatModel` subclass — about 40 lines — and you get *all* of the above for free, including `with_structured_output` and LangSmith tracing. `../code/fakes.py` does exactly that, which is why every example in this track runs offline.

4. LCEL: composition, not a DSL

```
chain = prompt | model | StrOutputParser()
```

`|` is `Runnable.__or__`, and it builds a `RunnableSequence`. Two more coercions matter:

- a **dict** in a chain position becomes a `RunnableParallel` (branches run concurrently);
- a **plain function** becomes a `RunnableLambda`.



The three plumbing primitives you will use constantly:

Primitive	Does	Use it when
<code>RunnableParallel(a=x, b=y)</code>	fan out, run branches concurrently, return a dict	two independent model calls, or retrieve-and-passthrough
<code>RunnablePassthrough()</code>	identity	you need the raw input to survive a parallel branch
<code>RunnablePassthrough.assign(k=f)</code>	add keys, keep the existing ones	carrying a question alongside retrieved context

LCEL's status in v1 — be precise about this

LCEL is **not deprecated** — `Runnable` is the substrate of the whole library. But the recommended way to build an *agent* is no longer a chain; it is `create_agent` on `LangGraph`, and the v1 docs no longer have an LCEL landing page at all. The honest split:

- **Fixed pipeline, known number of steps, no loops** → LCEL. Extraction, classification, summarisation, 2-step RAG. Declarative, streams well, traces cleanly.
- **The model decides what happens next, or there is a loop, or you need to pause/resume/persist** → `LangGraph` via `create_agent`. See `../langgraph/`.

People who tried to express an agent loop in LCEL discovered why `LangGraph` exists: a pipe operator has no vocabulary for cycles, checkpoints, interrupts, or "go back three steps."

5. Messages, prompts, parsers

Messages

Class	Role	Carries
<code>SystemMessage</code>	instructions	one per conversation, at the front
<code>HumanMessage</code>	user turn	text, images, files, audio
<code>AIMessage</code>	model turn	<code>.content</code> , <code>.content_blocks</code> , <code>.tool_calls</code> , <code>.usage_metadata</code>
<code>ToolMessage</code>	tool result	<code>.content</code> , <code>.tool_call_id</code> , <code>.status</code>

v1 added **standard content blocks**: `message.content_blocks` gives you reasoning traces, citations and server-side tool results in a provider-agnostic shape, while `.content` keeps working. Use `.content_blocks` in new code if you touch anything beyond plain text.

The invariant that bites everyone: every `tool_call` in an `AIMessage` must be answered by a `ToolMessage` with a matching `tool_call_id` before the next model call. Providers reject the conversation otherwise.

`../code/tools_and_structured_output.py` demonstrates this and the three ways it goes wrong.

Prompt templates

```
ChatPromptTemplate.from_messages([
    ("system", "You are a {persona}."),
    MessagesPlaceholder("history"),      # splice a whole list of messages in
    ("human", "{question}"),
])
```

Few-shot: `FewShotChatMessagePromptTemplate` with a fixed `examples` list, or with an `example_selector` (e.g. `SemanticSimilarityExampleSelector`) that picks examples per query from a vector store — a real technique that costs one embedding call per request.

Gotcha: `{` is a format character. Put raw JSON in a prompt literal and you get a `KeyError` on your own example payload. Escape `{{ / }}`, or pass the JSON in as a variable.

Output parsers, and why you probably want structured output instead

`StrOutputParser` (pulls `.content` out; streams) and `JsonOutputParser` (parses JSON; buffers) earn their place. Most of the rest — the ones that beg for a format in the prompt then regex the reply — are pre-tool-calling archaeology.

```
class Review(BaseModel):
    sentiment: Literal["pos", "neg"]
    score: int = Field(ge=1, le=5)

model.with_structured_output(Review).invoke("...")  # -> Review
```

`with_structured_output` is tool calling underneath (`bind_tools([Schema], tool_choice=Schema)` plus a parser), or the provider's native JSON-schema mode where one exists. Two consequences, both important:

1. **Pydantic constraints are validated after generation, not during it.** `ge=1` does not stop the model emitting `0`; it raises `ValidationError` when it does. Catch it.
2. **`include_raw=True`** returns `{"raw", "parsed", "parsing_error"}` instead of raising — which also gets you the token usage from the failed call. Use it in anything with a retry policy.

6. Tools

```
@tool
def get_seat_count(aircraft: str) -> int:
    """Return the number of passenger seats on an aircraft model.

    Args:
        aircraft: ICAO-ish model code, e.g. "A320".
    """
```

The decorator turns the function into a `BaseTool`: `name` from the function name, `description` from the docstring, `args_schema` from the type hints. **The docstring is a prompt**, and a vague one is a prompt bug that presents as "the model keeps picking the wrong tool."

`model.bind_tools([t1, t2])` returns a `RunnableBinding` — the same model with the tool schemas attached to every subsequent call. Not magic, just frozen kwargs.

The loop, in five steps: `bind` → model returns `AIMessage.tool_calls` → you execute → you append one `ToolMessage` per call → call the model again. `create_agent` / `ToolNode` runs this for you, but write it by hand once. Error handling belongs *inside* the loop:

Failure	Right response
Tool name doesn't exist (hallucinated)	<code>ToolMessage(status="error")</code> listing the real tools
Tool raises <code>ToolException</code>	<code>ToolMessage(status="error")</code> with the message; model retries or abstains
Args fail schema validation	<code>ToolMessage(status="error")</code> naming the offending fields
Tool hangs	timeout at the tool, not the request
Model loops forever	a hard <code>max_turns</code> guard. Non-negotiable.

A tool failure is context, not an exception. Crashing on a bad tool call converts a recoverable turn into a 500. In v1 the framework-level hook for this is `@wrap_tool_call` middleware on `create_agent`.

7. Retrieval, assembled

```
load → split → embed → store → retrieve → generate
|       |       |       |       |       |
Document Text Embeddings VectorStore Retriever ChatModel
Loader  Splitter
```

Piece	Interface	Notes
Document loader	<code>.load()</code> , <code>.lazy_load()</code>	use <code>lazy_load</code> on anything larger than RAM
Text splitter	<code>.split_documents(docs)</code>	<code>RecursiveCharacterTextSplitter</code> is the right default
Embeddings	<code>.embed_documents()</code> , <code>.embed_query()</code>	two methods, because some models embed queries differently
Vector store	<code>.add_documents()</code> , <code>.similarity_search()</code>	Chroma / FAISS / pgvector / Pinecone all implement it
Retriever	<code>.invoke(query) -> list[Document]</code>	it's a Runnable, so it drops straight into a chain

The canonical LCEL RAG chain — memorise this shape:

```
chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt | model | StrOutputParser()
)
```

Three facts that decide whether your RAG works:

- **A retriever always returns k results**, however bad. Nothing is "no match". Ask an out-of-corpus question and you get the k nearest irrelevant chunks and a confident answer built on them.
`../code/mini_rag_chain.py` demonstrates this happening on purpose.
- **Metadata filtering is the cheapest accuracy win available**, and tenant isolation belongs in the filter, never in the prompt.
- **Similarity scores are not probabilities** and are not comparable across embedding models.

Chunking strategy, hybrid search, reranking and RAG evaluation are Module 08: `../../../../08-rag/`.

8. What is in the second note

Memory and history management, streaming/callbacks/caching, the debugging playbook, cost control, and the decision of when to graduate to LangGraph all live in `02-practitioner-craft.md`. Read it before you ship anything.

9. Where this returns

Idea here	Where it returns
Runnable <code>le</code> , LCEL composition	<code>../langgraph/</code> — nodes are Runnables
Tool calling protocol	<code>../../../../06-ai-agents/</code> , <code>../../../../09-mcp/</code>
Retriever / vector store / chunking	<code>../../../../08-rag/</code>
Tracing, spans, cost attribution	<code>../langsmith/</code>
Structured output & schema validation	<code>../../../../05-genai/</code>
Failure taxonomy, abstention scoring	<code>../../../../15-ai-evals/</code>
Provider-neutral model interface	<code>../aws-bedrock/</code> , <code>../../../../11-llm-models/</code>
Prompt injection via retrieved docs	<code>../../../../13-ai-security/</code>